SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO 2_ASSIGNMENT

C++ Functions and Object-Oriented Concepts

Course Outcome: CO2 – Design C++ programs by applying functions and object-oriented concepts
Student Name: Keshavaram L
Register Number: 192511253
Department: B.E. Computer Science and Engineering
Academic Year: 2026–2027
GitHub Repository Link: https://github.com/keshavaram0104-boop/DSA0118_OOP-WITH-C-_192511253_KESHAVARAM-L

COURSE OUTCOME

CO2: Design C++ programs by applying functions and object-oriented concepts including function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, classes, member functions, access specifiers, arrays within a class, static data members, arrays of objects and friend functions.

ASSIGNMENT OVERVIEW

This report answers Q1–Q7 from Assignment 2. Each question includes the design explanation, complete C++ source, execution output and alternative-concept complexity analysis. Programs are written in standard C++17.

Q1 – Call by Value, by Reference and by Pointer

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

Call by value copies the fee, so the caller's variable remains unchanged. Call by reference aliases the caller's variable; a pointer also modifies the caller through dereferencing. bigger() returns a reference to the larger lvalue, so assignment through the call changes that original variable. show(1) uses y=5 and z=10; show(1,2) uses z=10; show(1,2,3) uses no defaults. For a fee update, returning a new value is often easier to reason about when multiple concessions are composed; reference mutation is efficient and suitable when in-place update is the required contract. All versions are O(1) time; passing a large object by value copies it, while reference and const reference avoid that copy.

SOURCE CODE

```
#include <iostream>
using namespace std;
void change(int a, int& b, int* c) { a += 10; b += 10; *c += 10; }
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { cout << x + y + z << ' '; }
void applyByValue(long long fee, int percent) { fee -= fee * percent / 100; }
void applyConcession(long long& fee, int percent) { fee -= fee * percent / 100; }
int main() {
 int a=1,b=2,c=3; change(a,b,&c); cout<<a<<' '<<b<<' '<<c<<'\\n';
 bigger(a,c)=100; cout<<a<<' '<<c<<'\\n';
 show(1); show(1,2); show(1,2,3); cout<<'\\n';
 long long fee=45000; applyByValue(fee,10);
 cout<<"after call by value : "<<fee<<'\\n';
 applyConcession(fee,10); cout<<"after call by reference: "<<fee<<'\\n';
}
```

OUTPUT

```
1 12 13236621 1002366216 13 6 23662after call by value : 4500023662after call by reference: 4050023662
```

COMPLEXITY ANALYSIS

Time: O(1) for each parameter-passing operation. Auxiliary space: O(1); a by-value large object adds a copy proportional to its size.

Q2 – Return by Reference and Its Dangers

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

Returning int& to an array element is safe while the array remains alive and lets the caller update the element. Returning a reference to a local variable is invalid because the local object is destroyed on return; using the dangling reference is undefined behaviour and compilers commonly warn about returning a reference to a local. A const reference prevents modification through that reference; returning a copy is safest when lifetime is uncertain. A vector growth operation may reallocate storage and invalidate previously obtained references and pointers. Finding the maximum is O(n) time and O(1) auxiliary space whether returning a reference, pointer or index. An index can be checked again after vector growth and is generally more robust than a pointer/reference.

SOURCE CODE

```
#include <iostream>
using namespace std;
int& highest(int a[], int n) {
 int index=0; for(int i=1;i<n;++i) if(a[i]>a[index]) index=i;
 return a[index];
}
int* highestPointer(int a[], int n) {
 int index=0; for(int i=1;i<n;++i) if(a[i]>a[index]) index=i;
 return &a[index];
}
int main() {
 int marks[5]={72,88,91,85,79};
 highest(marks,5)+=2;
 cout<<"Highest after moderation: "<<highest(marks,5)<<'\\n';
 *highestPointer(marks,5)+=1;
 cout<<"Highest after pointer update: "<<highest(marks,5)<<'\\n';
 int index=0; for(int i=1;i<5;++i) if(marks[i]>marks[index]) index=i;
 cout<<"Highest index: "<<index<<", mark: "<<marks[index]<<'\\n';
}
```

OUTPUT

```
Highest after moderation: 9323662Highest after pointer update: 9423662Highest index: 2, mark: 9423662
```

COMPLEXITY ANALYSIS

Maximum search: O(n) time and O(1) auxiliary space. Returning an index, pointer or reference does not change the search complexity.

Q3 – Default Arguments and Function Overloading

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

The original overload set is ambiguous because a one-argument call can match charge(double) exactly and also match the three-parameter overload using defaults. Remove the redundant one-argument overload and use one double overload with rightmost defaults, plus distinct overloads for integer grams and Parcel. Overload resolution prefers exact match, then promotion, then standard conversion. Default arguments must be supplied from the right because omitted arguments form a trailing sequence. The chosen overload is resolved at compile time and adds no run-time dispatch cost. A Parcel structure scales well when optional features grow, while overloads offer convenient interfaces for different input types.

SOURCE CODE

```
#include <iostream>
using namespace std;
struct Parcel { double kg; bool express; bool insured; };
double charge(double kg, bool express=false, bool insured=false) {
 double total=50.0 + kg*20.0;
 if(express) total+=30.0; if(insured) total+=15.0; return total;
}
double charge(int grams) { return 50.0 + (grams/1000.0)*20.0; }
double charge(Parcel p) { return charge(p.kg,p.express,p.insured); }
int main() {
 cout<<"charge(2): "<<charge(2)<<'\\n';
 cout<<"charge(2.0): "<<charge(2.0)<<'\\n';
 cout<<"charge(2.0f): "<<charge(static_cast<double>(2.0f))<<'\\n';
 cout<<"charge(2000 grams): "<<charge(2000)<<'\\n';
 cout<<"express: "<<charge(2.0,true)<<'\\n';
 cout<<"parcel: "<<charge(Parcel{1.5,true,true})<<'\\n';
}
```

OUTPUT

```
charge(2): 50.0423662charge(2.0): 9023662charge(2.0f): 9023662charge(2000 grams): 9023662express: 12023662parcel: 12523662
```

COMPLEXITY ANALYSIS

Overload selection occurs at compile time. Each charge calculation is O(1) time and O(1) local storage.

Q4 – Specifying a Class with an Array Inside It

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

SavingsAccount keeps account number, balance, transaction array and count private. deposit() and withdraw() validate amounts and update the balance; miniStatement() prints the retained transactions. The array is private so callers cannot corrupt the statement history or balance. When the 11th transaction arrives, shifting discards the oldest and costs O(k) for k stored entries; a circular buffer uses O(1) insertion with a start index. Increasing capacity from 10 to 50 changes the fixed array size and bounds, while a deque or circular buffer handles the rolling history more naturally.

SOURCE CODE

```
#include <iostream>
#include <string>
using namespace std;
class SavingsAccount {
   string accountNumber; long long balance;
   long long transactions[10]; int count;
public:
   SavingsAccount(string number,long long
opening):accountNumber(number),balance(opening),count(0){}
   void deposit(long long amount);
   bool withdraw(long long amount);
   void miniStatement() const;
};
void SavingsAccount::deposit(long long amount) {
 if(amount<=0) return; balance+=amount;
 if(count<10) transactions[count++]=amount;
 else { for(int i=1;i<10;++i) transactions[i-1]=transactions[i]; transactions[9]=amount; }
}
bool SavingsAccount::withdraw(long long amount) {
 if(amount<=0 || amount>balance) return false; balance-=amount;
 long long entry=-amount;
 if(count<10) transactions[count++]=entry;
```

```
 else { for(int i=1;i<10;++i) transactions[i-1]=transactions[i]; transactions[9]=entry; }
 return true;
}
void SavingsAccount::miniStatement() const {
 cout<<"Account: "<<accountNumber<<" Balance: "<<balance<<" paise\\nRecent transactions
(paise): ";
 for(int i=0;i<count;++i) cout<<transactions[i]<<' ';
 cout<<'\\n';
}
int main() {
 SavingsAccount a("SA102",10000); a.deposit(2500); a.withdraw(1200);
 for(int i=0;i<10;++i) a.deposit(100*(i+1));
 a.miniStatement();
}
```

OUTPUT

```
Account: SA102 Balance: 16800 paise\nRecent transactions (paise): 100 200 300 400 500 600 700
800 900 1000 23662
```

COMPLEXITY ANALYSIS

A fixed-array shift is O(k) per new transaction; a circular buffer or deque end operations are O(1) amortized. Storage is O(k).

Q5 – Static Members and Arrays of Objects

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

The issued counter is static because it belongs to the class rather than one ticket. Its out-of-class definition allocates the shared storage. A static member function has no this pointer and cannot directly access non-static members. The Token constructor runs four times: three array elements and one extra object, so IDs are 101, 103 and 104 at the printed positions and issued() returns four. The BusTicket array creates four objects; only three seats are issued and the fourth waits. A global counter has similar O(1) time and one-counter storage but weaker encapsulation. A TicketOffice object supports multiple buses independently, which a single static counter does not.

SOURCE CODE

```
#include <iostream>
#include <string>
using namespace std;
class BusTicket {
   static int issued; static const int CAPACITY=3;
   int seat; string name;
```

```
public:
   explicit BusTicket(const string& n):seat(0),name(n) {
     if(issued<CAPACITY) seat=++issued;
   }
   static int count(){ return issued; }
   void print() const {
     cout<<name<<": "<<(seat ? "Seat "+to_string(seat) : string("Waiting"))<<'\\n';
   }
};
int BusTicket::issued=0;
class Token {
 static int next; int id;
public: Token(){id=++next;} int get()const{return id;}
 static int issued(){return next-100;}
};
int Token::next=100;
int main(){
 Token counter[3]; Token extra;
 cout<<counter[0].get()<<' '<<counter[2].get()<<' '<<extra.get()<<' '<<Token::issued()<<'\\n';
 BusTicket trip[4]={BusTicket("Asha"),BusTicket("Bala"),BusTicket("Chitra"),BusTicket("Dev")};
 for(const BusTicket& t:trip)t.print();
 cout<<"Issued: "<<BusTicket::count()<<'\\n';
}
```

OUTPUT

```
101 103 104 423662Asha: Seat 123662Bala: Seat 223662Chitra: Seat 323662Dev: Waiting23662Issued: 323662
```

COMPLEXITY ANALYSIS

Ticket construction and counter access are O(1). The shared counter uses O(1) storage; the ticket array uses O(n) objects.

Q6 – Friend Functions and Inline Functions

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

A friend function is appropriate when an operation compares two Time objects symmetrically. It is not a member and is called like an ordinary function, but it can access private h and m because the class grants friendship. minutesBetween converts each time to minutes after midnight and returns the absolute difference. earlier compares hour first and minute second. Small functions may be defined inline, but inline is a request/ODR facility rather than a guarantee that the compiler substitutes the

body. Member functions or public getters can provide similar O(1) behaviour; getters generally preserve encapsulation better if the comparison does not need privileged access.

SOURCE CODE

```
#include <iostream>
using namespace std;
class Time {
 int h,m;
public:
 Time(int hh,int mm):h(hh),m(mm){}
 friend int minutesBetween(const Time&,const Time&);
 friend bool earlier(const Time&,const Time&);
};
int minutesBetween(const Time& a,const Time& b) {
 int x=a.h*60+a.m, y=b.h*60+b.m; return x>y?x-y:y-x;
}
bool earlier(const Time& a,const Time& b) {
 return a.h<b.h || (a.h==b.h && a.m<b.m);
}
int main(){
 Time departure(9,45),arrival(13,10);
 cout<<minutesBetween(departure,arrival)<<' '<<boolalpha<<earlier(arrival,departure)<<'\\n';
 Time noonA(11,55),noonB(12,5);
 cout<<minutesBetween(noonA,noonB)<<' '<<earlier(noonA,noonB)<<'\\n';
 cout<<minutesBetween(noonA,noonA)<<' '<<earlier(noonA,noonA)<<'\\n';
}
```

OUTPUT

```
205 false2366210 true236620 false23662
```

COMPLEXITY ANALYSIS

Time comparison is O(1) and uses O(1) auxiliary space. Inline expansion may reduce call overhead but must be measured under the selected compiler and optimization flags.

Q7 – Diagnose, Redesign and Validate

AIM

To implement and evaluate the C++ concept specified in this question using a complete executable program.

EXPLANATION AND DESIGN DECISION

The fixed register stores only ten marks, so an 11th record cannot be accepted. The measurable objective is to support a growing number of marks without changing a compile-time array bound. The redesigned class uses std::vector<int>, which grows dynamically and retains a simple class interface. The vector uses additional capacity-managed memory; append is amortized O(1), and maximum search is O(n). The fixed array has predictable memory and no allocation overhead when the maximum size is known. The dynamic design is preferable when the number of records is not known in advance. Validation uses the same initial values plus additional records and confirms the dynamic register stores twelve entries and reports the maximum.

SOURCE CODE

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
class FixedRegister {
 int marks[10]; int count;
public:
 FixedRegister():count(0){}
 void add(int x){if(count<10)marks[count++]=x;}
 int maximum()const{int m=marks[0];for(int i=1;i<count;++i)if(marks[i]>m)m=marks[i];return m;}
};
class DynamicRegister {
 vector<int> marks;
public:
 void add(int x){marks.push_back(x);}
 int maximum()const{int m=marks[0];for(int x:marks)if(x>m)m=x;return m;}
 size_t size()const{return marks.size();}
};
int main(){
 FixedRegister fixed; for(int x:{72,88,91,85,79})fixed.add(x);
 DynamicRegister dynamic; for(int x:{72,88,91,85,79,94,67,90,86,93,98,81})dynamic.add(x);
 cout<<"Fixed register maximum: "<<fixed.maximum()<<'\\n';
 cout<<"Dynamic register count: "<<dynamic.size()<<'\\n';
 cout<<"Dynamic register maximum: "<<dynamic.maximum()<<'\\n';
 cout<<"Redesign: dynamic storage accepts records beyond the fixed capacity.\\n";
}
```

OUTPUT

```
Fixed register maximum: 9123662Dynamic register count: 1223662Dynamic register maximum: 9823662Redesign: dynamic storage accepts records beyond the fixed capacity.\n
```

COMPLEXITY ANALYSIS

Adding an element to vector is amortized O(1); finding the maximum is O(n). Storage is O(n), compared with O(1) fixed capacity for the ten-element example.

FINAL CONCLUSION

The assignment demonstrates parameter passing, safe reference use, function overloading, default arguments, class encapsulation, arrays within classes, static members, friend functions and inline functions. The redesigned dynamic register illustrates how a class can evolve when fixed-capacity assumptions no longer fit the requirement. Add the working GitHub repository URL, screenshots and timing measurements from your own machine before final submission.